

SpringJDBC_Day02

How the JDBCTemplate and NameParameterJdbcTemplate works

so suppose we have a product table where i have to perform some operations in database for this table

- insert
- delete
- update
- fetch [curd operations]

generally w'll not perform any operation through the application as

- create table
- commit or rollback
- transactions

so for performing these operation earlier we are using JDBC API

Product Table

- Product No
- Product Name
- About
- Price

operations

so for this we should have to write one class and inside the class we have to write some logic which can connect to the database and can able to perform some operations

```
class Main{
    public static void main(String [] args)
    {
        1. Load the driver
        2. Create the connection
        3. We have to write the query to perform the curd operations
        4. create the statements
        5. Create the resultset
        etc
    }
}
```

so through this class i can connect with the database and able to perform the operations in the database table

- so that type of class is known as DAO

Spring JDBC TELLS US THAT DON'T WRITE THESE THINGS BY OUR OWN I WILL MANAGE IT JUST USE THE CONTENT WHICH I WILL PROVIDE

- JdbcTemplate
- NamedParameterJdbcTemplate

These two are the classes provided by the spring jdbc now let's see how it's work

Product Dao class

- JdbcTemplate template;

DataSource

DataSource [interface]

- DriverManagerDataSource [class]

- several more class
- Hikaricp
- etc

DataSource is an interface which is present in the package javax.sql.DataSource

- It is used for the Connection pooling purpose

so we cannot create the object of the interface that's why spring jdbc had provided us some implementation class so that we can create the object and call the methods which perform the operations

so changing one class into another class can cause issue for performing the operation that's why jdbcTemplate use the DataSource interanally and it call the DriverManagerDataSource for the implementation

Let's see the example that how actually it's work

```
class ProductDao{
    JdbcTemplate template;
    /* so suppose if i want to perform some operation in db i need all the configuration with the database ( driver, connection )
    ✓
    ProductDao(JdbcTemplate template)
    {
        this.template=template;
    }
}
```

interanally it will use the interface

DataSource

- whose implementation class is DriverManagerDataSource

so we are not manually creating the object we will tell the spring to manage all these creation, dependency and distructions

So How many approach we know to do all these

- XML
- XML+Annotation
- Annotation
- Java Configuration approach

so among them we are going to use

- Annotation
- JavaConfiguration approach

- with the xml approach

```
<beans>
<bean id="datasource" package="datasource">
    <property name="driver" value="oracle.jdbc.OracleDriver"/>
    <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>
    <property name="username" value="system"/>
    <property name="password" value="manager"/>
</bean>

<bean id="jdbcTemplate" class="packageName.JdbcTemplate">
    <constructor ref="datasource"/>
</bean>

<bean id="productDao" class="packageName.ProductDao">
    <constructor ref="jdbcTemplate"/>
</bean>
</beans>
```

let's see our Main requirement of the day

Product Table

- Product No
- Product Name
- About
- Price

```
class Product {
    private static final SQL_COUNT_OF_PRODUCT
    = "Select count() from Product";
    private JdbcTemplate template;
```

```
    Product(JdbcTemplate template)
    {
        this.template=template;
    }
```

```
    public int getNoOfProduct(SQL_COUNT_OF_PRODUCT)
    {
        return template.queryForObject(SQL_COUNT_OF_PRODUCT"
        Integer.class);
    }
```

SO FOR THIS OPERATION we are using

Class.forName()

Connection con=DriverManager

.getConnection();

PreparedStatement pstmt=

con.prepareStatement(Query);

ResultSet rs=pstmt.executeUpdate();